

17-214/514: Agentic Software Development

Communicating and Recording Designs

Hammad Ahmad and Claire Le Goues • Monday, September 14, 2026 • **Lecture**

Administrivia

- Midterm 1 is Wednesday, September 23, in class. It covers everything through next Monday's lecture.
- A2 checkpoint is due tomorrow (11:59pm).
 - Reminder: No late days for checkpoints!
- Window for A1 verbal defenses has been extended to also include tomorrow (9/15) office hour sessions.

Some things we saw in A1

- All submissions passed our hidden API checks. Many apps were still broken for a person using them.
 - Generally, a green API test suite may not say much about the client.
- Plans and logs written after the work, by the agent, in your voice.
 - From A2 on, commit your plans as you go. The Process points are for a history that shows it, and we spot-check timestamps.
- Transcripts are files, exported as you go.
 - Notably, not links to your sessions. Not one session for two weeks. Not a shell script that approves everything overnight.

What went right: many of you handled reconnects, the hardest repair. Also, you found about a dozen real bugs we had not listed.

What did we talk about last time?

- A design is the durable shape of a system (data, operations, structure), not its syntax.
- Design is assigning responsibilities and drawing boundaries, at every scale.
- A low representational gap matters more with agents, because they tend to amplify whatever you hand them.
- There's no correct design, but many wrong ones. Choose deliberately, and know what you traded.
- The design decisions stay yours. Getting them right makes generation tractable.
- So far, those decisions live only in your head.

Learning goals for today

- Record and justify a design decision so the **why** survives
- Use lightweight notation (sequence diagrams, sketches) to reason about a system
- Explain how design artifacts double as durable context for coding agents
- Describe what design patterns are, and why they're a communication tool

You made a great decision three weeks ago

Where is it now?

The code

The what survives:
an overlap check in `book()`

Not the why, not the rejected option

The agent's context

The whole debate happened here:
alternatives, constraints, reasons

Window closed, context gone

Your head

"we had a good reason...
something about the database?"

Six months of lossy compression

Where designs live

The design nobody wrote down

- The design exists, in one person's head.
- They leave (graduate, switch teams, forget their own reasoning).
 - The **what** stays in the code. The **why** leaves with them.
- Months later, nobody touches the weird part, because nobody remembers why it's weird.



programmerhumor.io

Software outlives the decisions that made it

- Code can live for years. The reasoning behind it fades in weeks.
- "Why is it built this way?" is an expensive question.



The agent wasn't in the meeting. (And it can't read your mind.)

**It knows only what you put in front
of it.**

Who reads a recorded design?

READER 1

Future-you

Six months out, you won't remember your own reasoning.

READER 2

The teammate

Whoever inherits the code or has to work with it. (In this course, the TA grading your rationale.)

READER 3

The agent

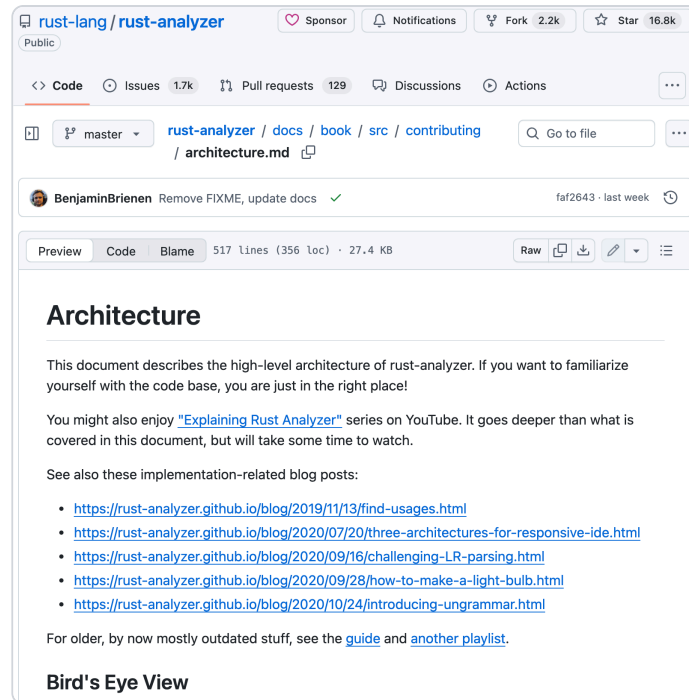
Starts from zero **every session**.

The design document

- A few pages on what the parts are, what each is responsible for, and **why these parts**.
 - Data, operations, structure, in writing.
- Short enough that people read it.
- A decision too big for a paragraph gets its own record.

ARCHITECTURE.md

- The repo's front door. The major parts, where things live, how to build and test.
- Written for a reader with zero context, human or agent.
- Later this semester you'll be dropped into a large codebase. You'll notice whether it has one.
- Structure rules in `ARCHITECTURE.md` are good candidates for architecture tests.



<https://github.com/rust-lang/rust-analyzer/blob/master/docs/book/src/contributing/architecture.md>

Recording decisions

What's an ADR?

An Architecture Decision Record is a short document that captures **one** decision.

- The context (what forced a choice),
- the decision (what you chose),
- the alternatives (what you rejected, and why),
- the consequences (what you bought, and what you gave up).

Short. One decision per file. Append-only. You don't edit history, you supersede it.

Anatomy of an ADR

ADR 007: Prevent double-booking with an overlap check in the domain

Status: Accepted

Context: A room must never hold two overlapping bookings. The check could live in the database (an exclusion constraint) or in the BookingService before insert.

Decision: Enforce it in BookingService.book(): reject the request if any existing booking on the room overlaps the requested interval.

Alternatives: A database exclusion constraint. Rejected because it ties the rule to one storage engine and hides it from the tests.

Consequences:

- + One clear, testable place for the rule.
- + No database-specific features required.
- Two concurrent requests can still race (revisit under concurrency).

Why bother?

- The code shows the overlap check. It doesn't say why it's there, or that a database constraint was considered.
- The record keeps the rejected alternative and the reason. That's the part one easily forgets.
- When an assumption breaks (two requests race), the record tells you which decision to revisit.
- Some decisions have no home in the code at all.
 - "The API layer never touches storage directly" lives in every file and is explained in none.

Write the minus signs down

- Every decision buys something **and** costs something. Both go in the record.
 - ADR 007's minus line ("concurrent requests can still race") comes back later in the course.
- A consequences section with no downsides isn't really a design decision.

"Self-documenting code" isn't enough

- Good names show what the code does.
- They don't say why, or what you rejected.
- The why needs prose, somewhere findable.

Let's practice: review this ADR

It has all four parts. You're the reviewer. Thoughts?

ADR 011: Store bookings in SQLite

Status: Accepted

Context: Bookings have to survive a restart. SQLite is a file-based database that needs no server and ships with most languages.

Decision: Use SQLite, behind a BookingStore interface so the domain never sees SQL.

Alternatives: Keep bookings in memory. Rejected because they would be lost on restart.

Consequences:

- + No database server to run or configure.
- + The BookingStore interface keeps SQL out of the domain.
- A little more setup than an in-memory list.

What's wrong with ADR 011?

- The alternative was never a real option. Why not a server database?
- The listed minus is minor. The real one (one file, one writer, one machine) is missing.
- The context describes SQLite, not the problem. What scale, how many writers, where does it run?
- Two decisions with different lifetimes in one file. The interface may well outlive SQLite. Superseding 011 buries it.

Which decisions get a record?

- Expensive to reverse.
Surprising. Cross-cutting.
Anything you argued about.
- Not the loop variable's name.
- In this course, a few good records are better than a pile of boilerplate ones.
- Rule of thumb: if a reviewer might ask "why?", and the answer is not "immediately obvious", write it down.

Scope of the ADR process

Project members should create an ADR for every architecturally significant decision that affects the software project or product, including the following (Richards and Ford 2020):

- Structure (for example, patterns such as microservices)
- Non-functional requirements (security, high availability, and fault tolerance)
- Dependencies (coupling of components)
- Interfaces (APIs and published contracts)
- Construction techniques (libraries, frameworks, tools, and processes)

docs.aws.amazon.com, AWS Prescriptive Guidance

For agentic SE, a decision log is key engineering memory

- A dated, append-only log of **why the system is the way it is**.
 - Read by new teammates, future-you, and agents.
- Cheap insurance against "why is it like this?"

Name	Last commit message	Last commit date
0001-record-architecture-decisions.md	Add ADR 001 and ADR 002: Minimum supported Python ver...	7 years ago
0002-minimum-supported-python-version.md	Add ADR-0020: Minimum supported Python version (#911)	3 years ago
0003-monitor-condition-and-data-selectors....	Fix broken link to dev docs. (#514)	5 years ago
0004-webscraping.md	Amend ADR004 to allow extracting forms during the auth ph...	5 years ago
0005-code-formatting.md	Update ADR-0005, add isort (#324)	7 years ago
0006-docker-images.md	Docker images (#265)	7 years ago
0007-integration-config-yaml-structure.md	Add exception for unique_id to ADR0007 (#718)	4 years ago
: nine more records		
0017-hardware-screening-os.md	Hardware screening OS (#452)	5 years ago
0018-supported-databases.md	Update supported databases and EoL policies (#1462)	18 hours ago
0019-GPIO.md	Update ADR-0019 and ADR-0020: remove Supervised and ...	6 months ago
0020-minimum-supported-python-version.md	Update ADR-0019 and ADR-0020: remove Supervised and ...	6 months ago
0021-YAML-integration-configuration-depreca...	Add ADR-0021: YAML integration configuration deprecation ...	3 years ago
0022-integration-quality-scale.md	Add ADR-0022 - Integration Quality Scale (#1162)	2 years ago

<https://github.com/home-assistant/architecture/tree/master/adr>

Never rewrite an accepted decision

- When a decision changes, add a **new** record and mark the old one superseded.
- Editing the old record loses the trail.
- In this course, from A3 on, a change that touches an old decision cites it and confirms or supersedes it.

ADR 004: in-memory store • superseded

->

ADR 011: SQLite • superseded

->

ADR 019: Postgres • accepted

Supersession in the wild

2. Minimum supported Python version

Date: 2019-05-13, updated 2023-02-05 (Python version clarification)

Architecture issue: [#167](#), [#278](#)

Status

Superseded by [ADR-0020](#)

0020. Minimum supported Python version

Date: 2023-05-29

Architecture discussion: [#903](#)

Replaces: [ADR-0002](#)

Changelog:

- 2026-04-02 Removed Supervised and Core from the list of installation methods. Removed Core-specific deprecation period policy.

Status

Accepted

<https://github.com/home-assistant/architecture/tree/master/adr> (records 0002 and 0020)

The format is negotiable

- The "A" was for architecture. Nygard coined the form in 2011 for structural decisions.
- Teams now use it for any decision worth explaining. ADR 007 is about one rule in one service, not the shape of the system.
 - With an agent reading the log too, more decisions are worth explaining.
- A design doc, an RFC, a `decisions.md`, or a thorough PR description can do the same job.
- What's required in agentic SE: the reasoning survives the session it was made in.

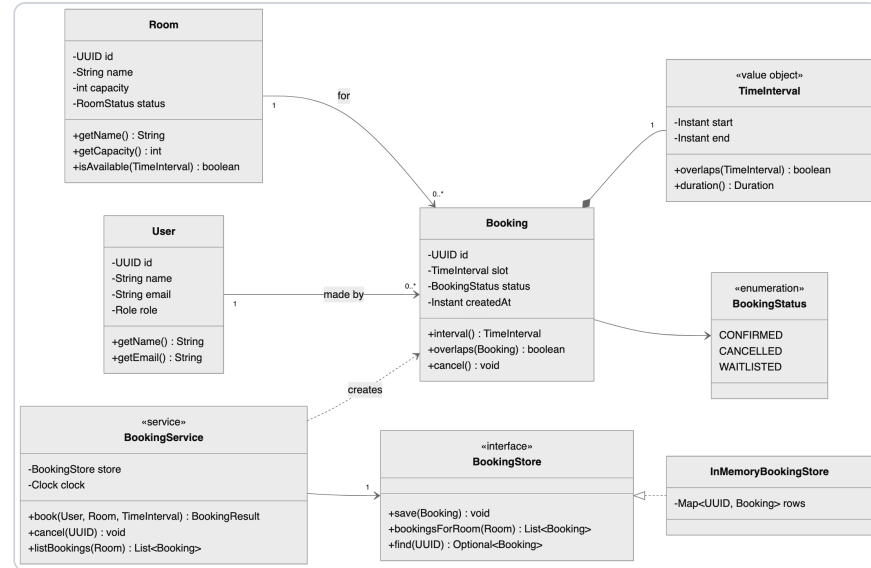
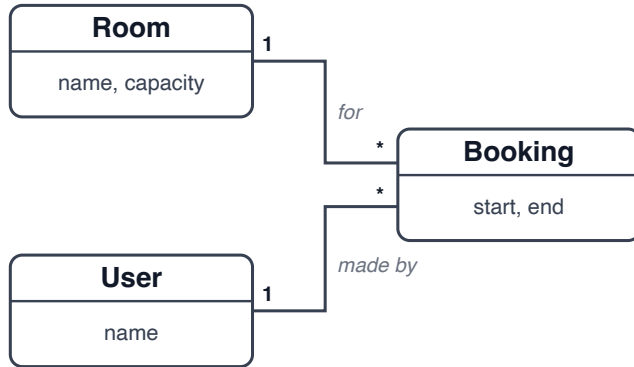
Your decision log starts this week

- A2 requires four founding decision records.
 - Data model (with a diagram), capability designs, storage, stack.
- Tomorrow night's checkpoint is `CAPABILITY.md` and your harness. The records are due with the assignment on the 21st.
- One evolving `decisions/` log, from now until December.
- Later assignments build on this system and its log.

Lightweight notation

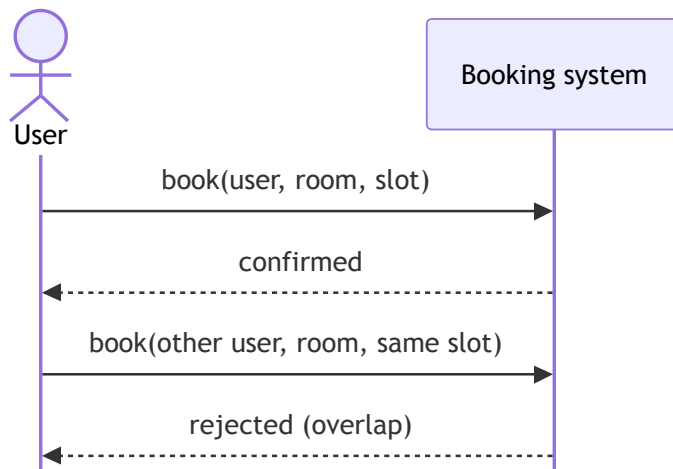
You don't need full UML

- Draw the one picture that answers your question.
- The goal is shared understanding.



System sequence diagrams

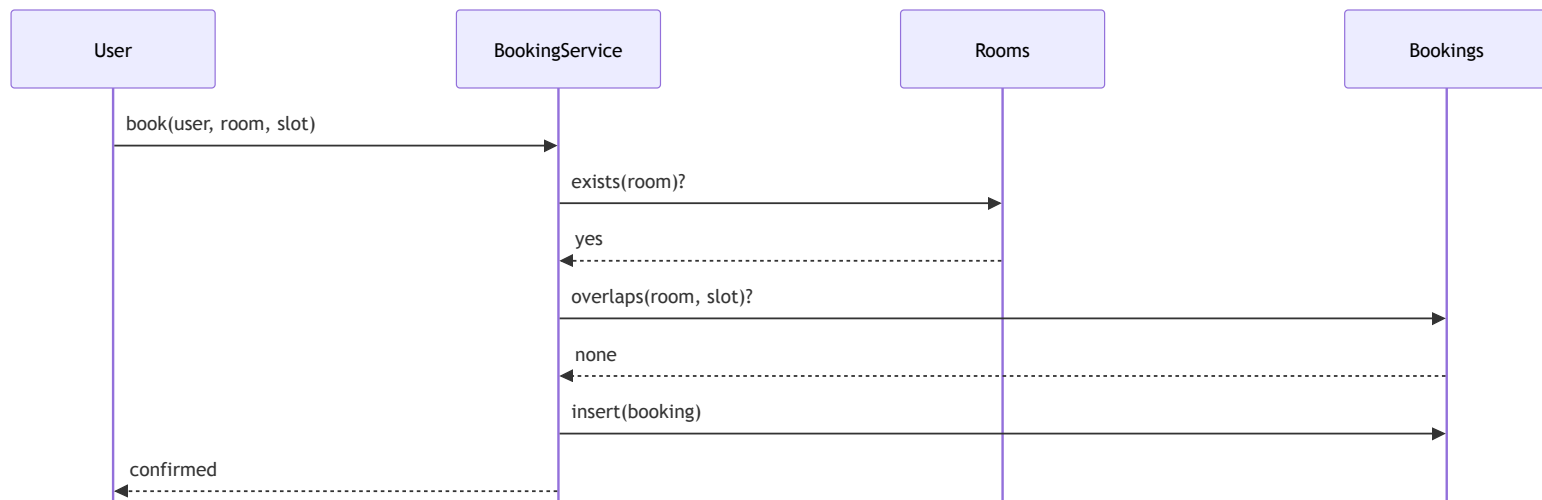
The user and the system, as a black box. What goes in, what comes back, in what order.



Each row translates nicely to a test case. Your A2 harness needs many more rows than this, but corresponds approximately to this shape.

Object interaction diagrams

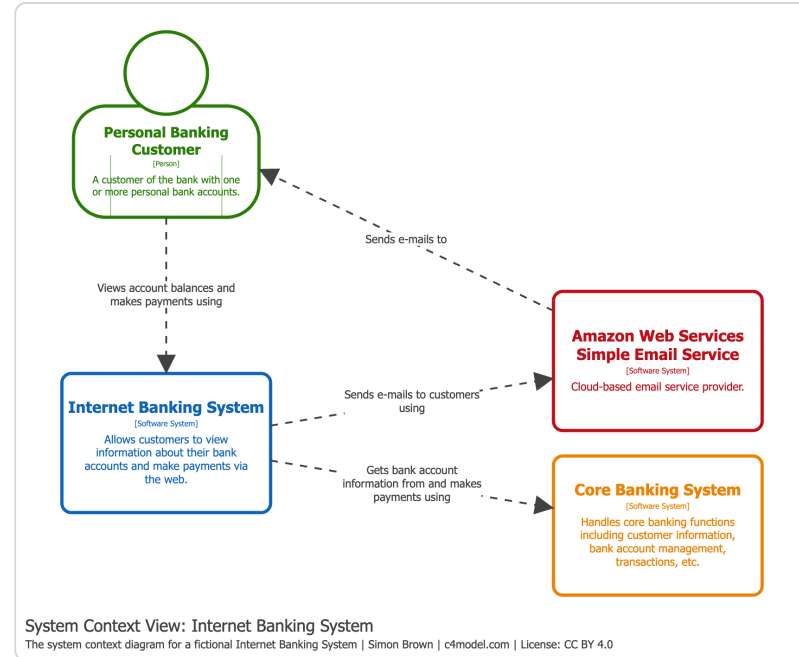
The same `book()` call inside the system. Who calls whom, in what order. (Same sequence-diagram notation, now the objects inside.) You'll mostly read these, not draw them.



The overlap check happens **before** the insert. What happens when it finds one?

Architecture sketches

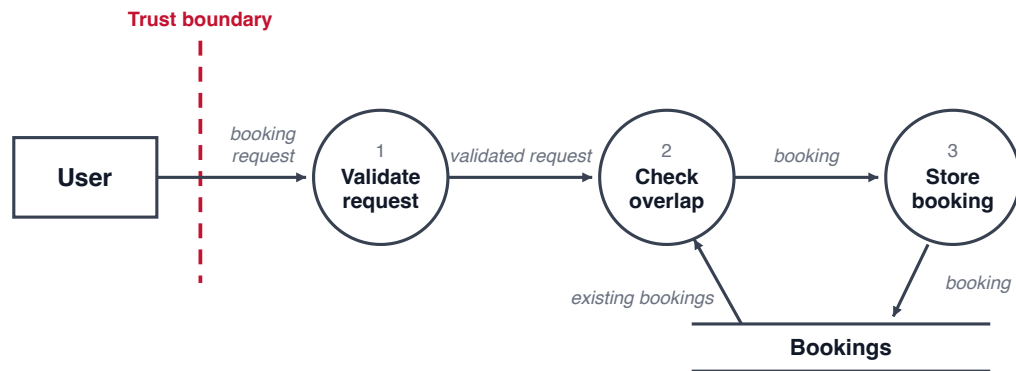
- Boxes for subsystems, arrows for who depends on whom.
- Answers "what are the major parts, and what talks to what?"
- If it doesn't fit on one slide, that tells you something about the system.



Simon Brown, c4model.com, CC BY 4.0

Dataflow diagrams

- Trace how one piece of data moves and gets transformed through the system.
- Four element kinds. Entities (rectangles), processes (circles), data stores (open boxes), flows (arrows). Every arrow is named for the data it carries.
- The data store is what a sequence diagram does not necessarily show.
- Good for spotting where data crosses a trust boundary (Week 13).



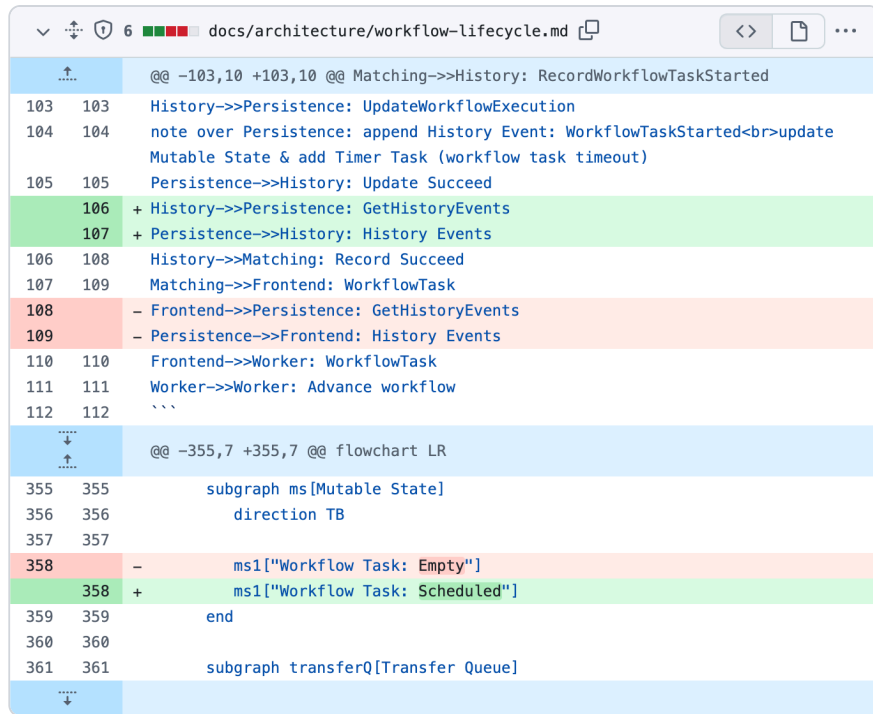
DeMarco/Yourdon notation. Gane-Sarson is the other common one (rounded processes, boxed stores, same four kinds).

Which notation?

- It depends. On the question you're asking.
- Some diagrams are for thinking. Draw (or have drawn), learn, discard.
- Some are for documenting. Those you keep, and keep current.

You can keep diagrams in the repo

- A screenshot on someone's laptop is already out of date.
- Text diagrams (Mermaid, PlantUML) live next to the code and diff in pull requests.
- An agent can read and update a text diagram. Not your whiteboard photo.



The screenshot shows a GitHub pull request diff for the file `docs/architecture/workflow-lifecycle.md`. The diff highlights changes to a Mermaid diagram. The top section shows a sequence of tasks and state updates, with line numbers 103 to 112. The bottom section shows a subgraph `ms[Mutable State]` with line numbers 355 to 361. The diff uses color coding: green for additions, red for deletions, and blue for context lines.

```
@@ -103,10 +103,10 @@ Matching-->>History: RecordWorkflowTaskStarted
103 103 History-->>Persistence: UpdateWorkflowExecution
104 104 note over Persistence: append History Event: WorkflowTaskStarted<br>update
105 105 Persistence-->>History: Update Succeed
106 106 + History-->>Persistence: GetHistoryEvents
107 107 + Persistence-->>History: History Events
108 108 History-->>Matching: Record Succeed
109 109 Matching-->>Frontend: WorkflowTask
110 110 - Frontend-->>Persistence: GetHistoryEvents
111 111 - Persistence-->>Frontend: History Events
112 112 Frontend-->>Worker: WorkflowTask
Worker-->>Worker: Advance workflow
```  


```
@@ -355,7 +355,7 @@ flowchart LR
355 355 subgraph ms[Mutable State]
356 356 direction TB
357 357
358 358 - ms1["Workflow Task: Empty"]
359 359 + ms1["Workflow Task: Scheduled"]
360 360 end
361 361 subgraph transferQ[Transfer Queue]
```


```

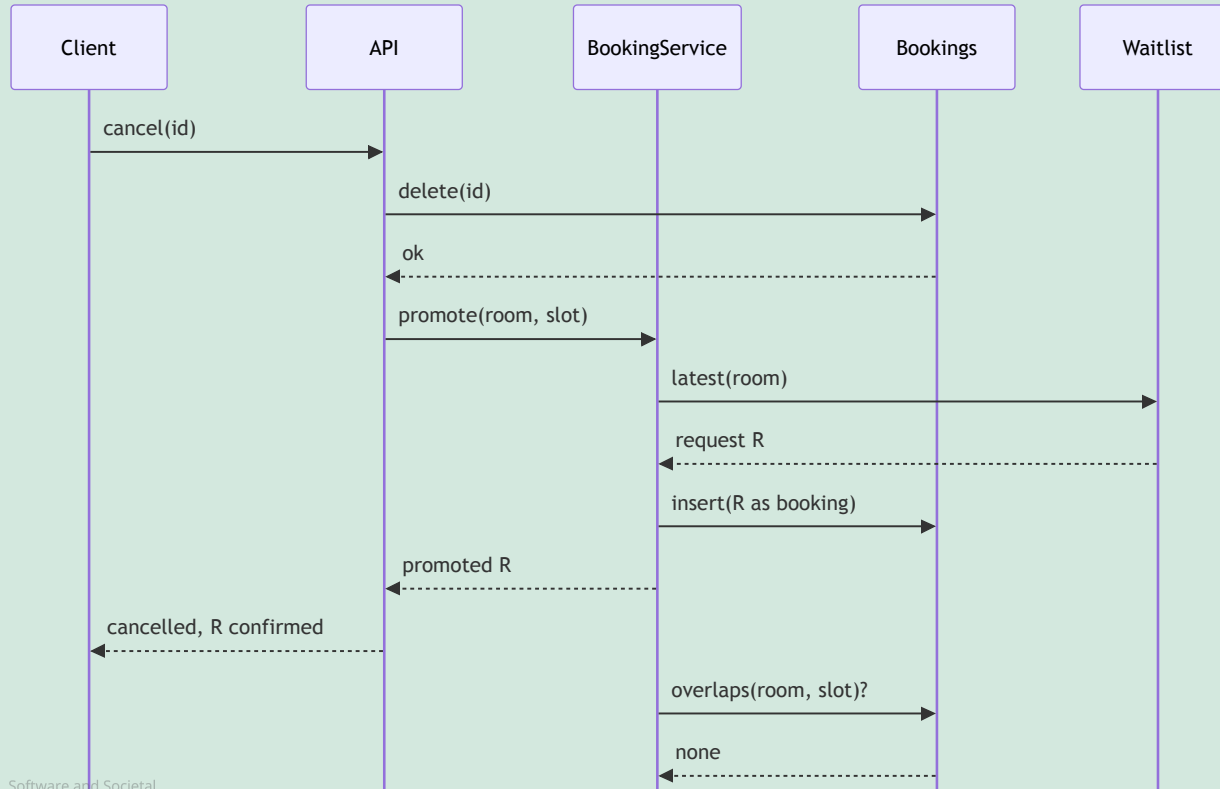
<https://github.com/temporalio/temporal/pull/9210/files>

# Let's practice: read this diagram

Here is the booking contract. The next slide shows the trace for `cancel()` and the promotion that follows. Where does the trace break the contract? What is missing?

1. A booking is confirmed only if nothing overlaps it in that room.
2. An overlapping request waits on the room's waitlist, in arrival order.
3. Cancelling promotes the **first** waiting request whose slot is now free.
4. Nothing is stored until its check has passed.
5. A reply reports what the store holds.
6. The API layer calls the service. The service calls the stores.

# The `cancel()` trace



Canvas → Quizzes  
→ Lecture 6  
In-Class Quiz  
Code: "sequence"

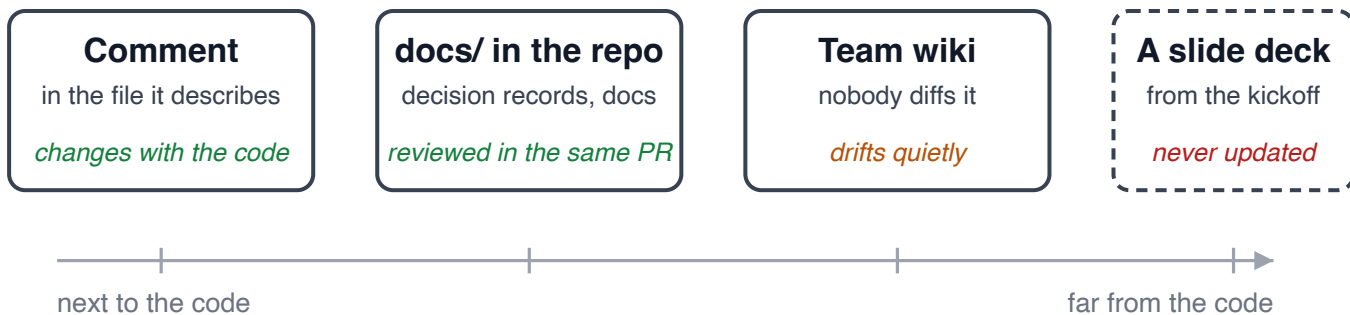
# What the diagram shows

- The insert happens before the overlap check. The check has to come first.
- Promotion takes the **latest** request. The contract says the first.
- The client hears "confirmed" before the check has run.
- The API layer deletes from the store directly. The service never sees the cancel.
- (And what it leaves out is the path where the check says **overlaps**. What happens to R?)

Which of these would your A2 harness catch?

# Keep it lightweight, keep it close

- Documentation lives next to the code it describes, or it rots.
- A stale doc is arguably worse than none.
- Write less. Keep it current. Put it where people and agents will find it.



# A README admitting defeat

grafana / README.md

 **yuriyDeneha** Minor change to run pipeline b37f474 · 2 years ago [History](#)

Preview Code Blame 147 lines (100 loc) · 4.99 KB Raw Copy Download Menu

## GROUNDWORK GRAFANA

**TODO: This readme is out of date, and mostly incorrect.**

Groundwork Grafana is built off the latest stable 5.4.x branch of Grafana. This branch enables an iframe substitution on links in the Grafana graph/dashboard share modal.

<https://github.com/gwos/grafana/blob/372cf315d1714a26276e6ab704771453d36779c7/README.md>

# What **not** to write down

- Not the obvious. The code already says `i++` .
  - **Do write:** The surprising, the decisions others depend on, and "we tried the obvious thing first."
- Something to keep in mind: every line written is a line someone has to keep true.

# Communicating to humans and agents

# Design artifacts are agent context

- Decision records, diagrams, and design docs are agent input.
- Hand them over so the agent works **with** your design.
- Onboarding a teammate and onboarding an agent are the same job.
  - An architecture sketch, the key decision records, an instruction file, how to run the tests.
- Write it once (and keep it up to date). Every later session reads it.

# Instruction files: CLAUDE.md / AGENTS.md

The file the agent reads at the start of **every** session.

```
AGENTS.md
- Architecture: storage / domain / api layers. Don't let api touch storage directly.
- The data model is in domain/models. Schema changes need a decision record.
- Run tests with `npm test`. Never commit with failing tests.
- Don't touch billing/. Ask a human.
```

## Git

This refines the Bash tool's `# Git` guidance to "branch first" when on the default branch:

- If HEAD is detached, that is intentional (the ghstack workflow). Do NOT create a new branch; commit directly onto the current detached HEAD.
- If you are on an actual branch (including `main`), follow the default guidance and branch first before committing.
- **Pulling CI status.** A PR has hundreds of check-runs, so a single `check-runs?per_page=100` call silently truncates and makes red look green. Use `gh pr checks <PR> --json name,state,workflow,link,bucket,completedAt` (already head-only, no paging).

<https://github.com/pytorch/pytorch/blob/main/CLAUDE.md>

# What belongs in one

- Conventions: how this codebase does things.
- Boundaries: what not to cross, what not to touch.
- How to build, test, and run.
- Invariants the agent must not break (especially around the data model).

Recall Week 1: this file is **read**, not necessarily obeyed. It describes the big picture. Tests and hooks enforce the parts that must hold.

# What's a skill?

A folder with a `SKILL.md` in it. Instructions the agent loads when a task matches.

```

name: pdf-processing
description: Extract text and tables from PDF files, fill forms, merge
 documents. Use when working with PDF files or when the user mentions PDFs.

PDF Processing
Use pdftplumber to extract text... For form filling, see FORMS.md.
```

- At startup the agent sees only the name and description. The body loads when a task matches it.
- Bundled files and scripts load only when the body points at them.
- An instruction file is always on. A skill is on demand.

# Superpowers

- Whole workflows get packaged as skills. Superpowers ships brainstorm, plan, TDD, and code review as skills for a dozen-plus agent tools.
- Think of it as an enforced version of the loop we saw in Week 1.
- Is it "free"?
  - Every activated skill is context you pay for, on every task.

How much of that process does your task need? It depends.

## The Basic Workflow

1. **brainstorming** - Activates before writing code. Refines rough ideas through questions, explores alternatives, presents design in sections for validation. Saves design document.
2. **using-git-worktrees** - Activates after design approval. Creates isolated workspace on new branch, runs project setup, verifies clean test baseline.
3. **writing-plans** - Activates with approved design. Breaks work into bite-sized tasks (2-5 minutes each). Every task has exact file paths, complete code, verification steps.
4. **subagent-driven-development** or **executing-plans** - Activates with plan. Dispatches fresh subagent per task with two-stage review (spec compliance, then code quality), or executes in batches with human checkpoints.
5. **test-driven-development** - Activates during implementation. Enforces RED-GREEN-REFACTOR: write failing test, watch it fail, write minimal code, watch it pass, commit. Deletes code written before tests.
6. **requesting-code-review** - Activates between tasks. Reviews against plan, reports issues by severity. Critical issues block progress.
7. **finishing-a-development-branch** - Activates when tasks complete. Verifies tests, presents options (merge/PR/keep/discard), cleans up worktree.

The agent checks for relevant skills before any task. Mandatory workflows, not suggestions.

<https://github.com/obra/superpowers>

# grill-me

A skill that interviews you about a design before anything gets built. From Skills for Real Engineers.

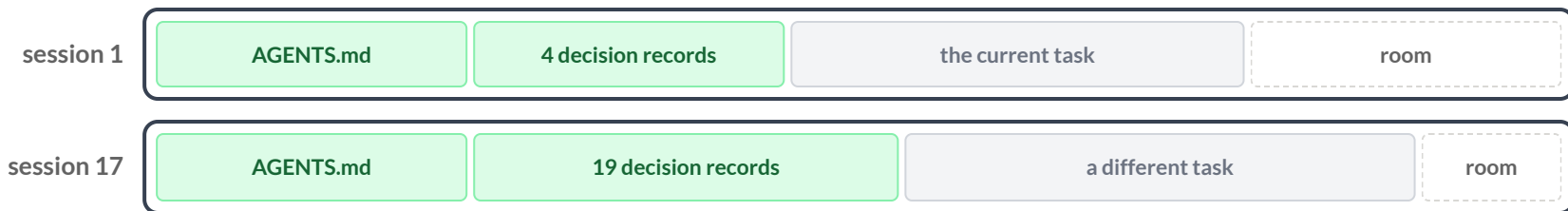
- It maps your plan as a tree of decisions and asks every question it can ask right now, each with a recommended answer.
- Facts are its job (it looks them up). Decisions are yours.
- It stops when nothing is left silently assumed, and waits for you to confirm before acting.

Interview the user relentlessly until you reach a shared understanding. [...]  
The decisions are the user's: put each to them and wait.

- The skill's own instructions.

# What survives between sessions?

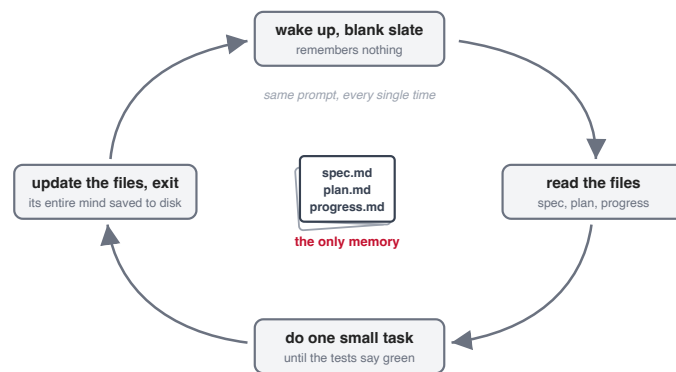
- Recall: the agent knows what's in its window and nothing else.
- Written artifacts are what **persists** across sessions.
- The idea: write it down once instead of explaining it every session.



The green files carry over from session to session, and they grow.

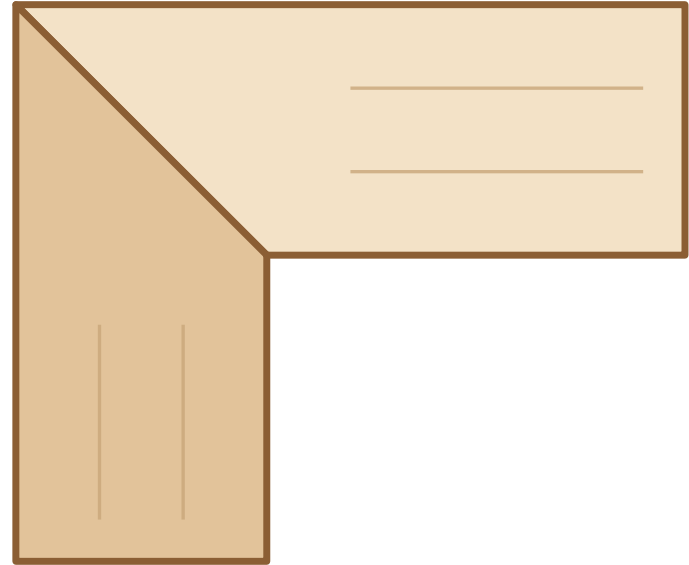
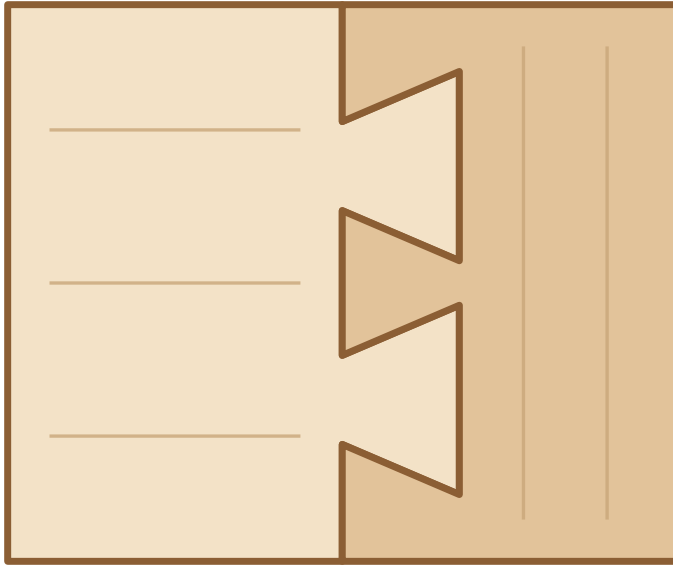
# Reminder: the Ralph Wiggum extreme

- A brute-force autonomous loop, fresh context every iteration.
- Its spec, plan, and progress files are its **entire** memory.
- The loop is only as good as those files.



# Patterns: a shared vocabulary

# How should we build these drawers?



# Two ways to answer

## Without a vocabulary

"Well, cut straight down into the wood, then back up at 45 degrees, then straight down again, then back up the other way at 45 degrees, then straight down, and repeat..."

## With one

"Use a dovetail joint."

Same conversation in software. "Write an if to handle..., then a while with a break so that..." versus "Strategy, or Template Method?"

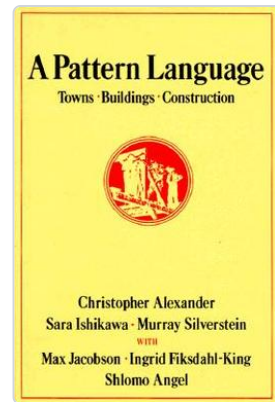
After Shalloway and Trott, Design Patterns Explained.

# What is a design pattern?

- A named solution to a recurring design problem.
- A **shape** with roles and relationships (not code to copy).
- "Strategy," "Observer," "Adapter." Each names a whole arrangement.

Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.

- Christopher Alexander, A Pattern Language (1977). An architect (of buildings, not software).



# Why do patterns exist?

- The same design problems recur across projects and decades.
- Patterns record the solutions so you don't rediscover them the hard way.
- These days, they're mostly **vocabulary**. The name lets two people (or a person and an agent) agree on a whole structure in one word.
  - That was the drawers. "Dovetail" is more efficient to communicate than the paragraph.
- Every pattern has four parts. A name, the problem, the shape of the solution, and the consequences.

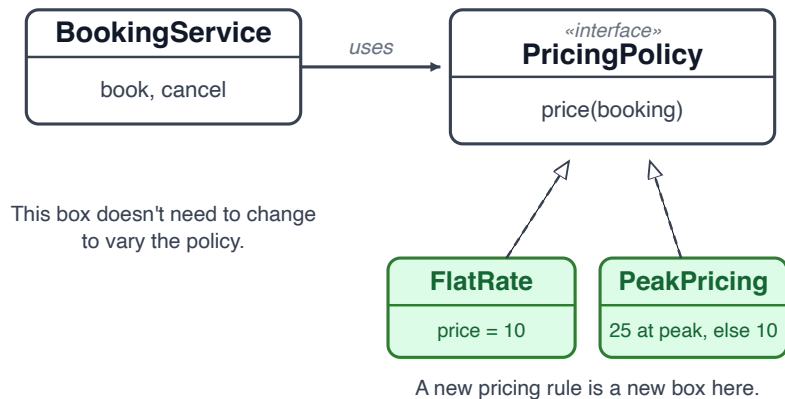
# A few you'll meet

- **Strategy:** swap an algorithm behind a common interface (think of a configurable sort order).
- **Observer:** let interested parties subscribe to events instead of constantly asking.
- **Adapter:** wrap a thing so it fits an interface it wasn't built for.
- **Factory:** centralize and name the messy work of creating objects.

More on these (and their misuse) in a couple of weeks. For now, each is a **name** for a structure.

# What Strategy looks like

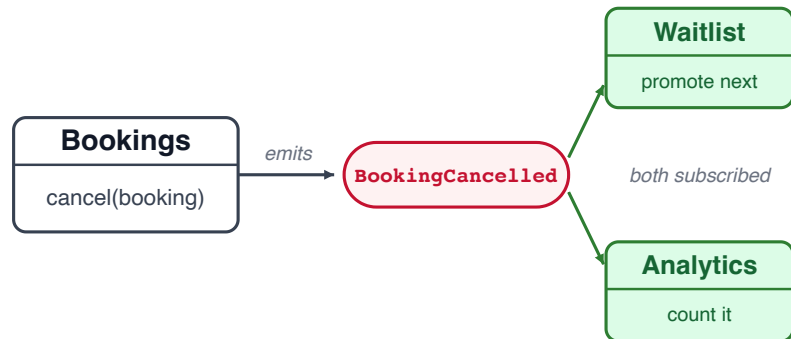
- The service depends on an interface, never on a particular rule.
- Each rule is its own box below.
- A new rule is a new box. The service doesn't need to change.
- Same notation as Wednesday's data model, plus one element (the interface). Read it the same way.



# What Observer looks like

A cancellation frees up a slot. The waitlist would very much like to know.

- Option A: the waitlist asks every minute. "Anything yet? Anything yet? Anything y..."
- Option B: bookings emit `BookingCancelled`. Interested parties subscribe.
- Option B is Observer. Adding a listener touches nothing on the left.



Bookings doesn't know who's listening.

Remember `BookingCancelled`. Events come back when we scale up.

# Patterns are a communication tool

- "Put a Strategy here" conveys the whole picture.
- To a teammate, a precise instruction.
- To an **agent**, the same, and more token-efficient. A well-known pattern is already in its training data. The name is the shortest way to point at it.
- It cuts both ways. The agent reaches for its favorite patterns unprompted, too.
  - Knowing its current defaults, and saying "not that one here," is part of the job. (More in a couple of weeks.)

# A caution

The ask: "Email the user when a booking is confirmed."

What came back:

```
src/notifications/NotificationStrategy.ts
src/notifications/EmailNotificationStrategy.ts
src/notifications/NotificationStrategyFactory.ts
src/notifications/BookingConfirmedObserver.ts
src/notifications/NotificationService.ts // a singleton
```

What it needed:

```
mailer.send(
 confirmation(booking));
```

- Patterns are vocabulary, not goals.
- Pattern soup is its own smell. Every file above is a real pattern. None of them answers a need.

# Summary

- A design in your head is lost with the context. Write it down.
- Decision records capture the **why**. The decision, the rejected options, the tradeoff. ADRs are one format. The function is what matters.
- Lightweight notation is for **understanding**. Learn to read it.
- Design artifacts double as agent context. Instruction files are the part the agent reads every session.
- Patterns are shared vocabulary, for humans and agents.

# What's next



- **Tuesday:** A2 checkpoint, 11:59pm. `CAPABILITY.md` and a harness covering it and the core contract.
- **Wednesday:** Design for change. Coupling, cohesion, and what a requirement change costs.
- **Friday, Lab 4:** Your first AWS deploy. Kept small on purpose.
- **Monday next week:** A2 due, 11:59pm, four founding records included.